

Class — groups — data → called as
attributes
 ↓ functions
 ↓ called as
 methods

defining class $\rightarrow$ class $\rightarrow$ name

eg: Class — :
pass

eg: class employee :
pass

Issues with manually writing data $\rightarrow$ prone to mistakes, that's

defining init :

Class employee:
def __init__():

self → using self in
init method points
current instance ~~variable~~
once execution is done,
self becomes next instance
~~variable~~.

def other
method instead
of writing it
manually

→ written inside
class

→ class employee:

def __init__():

eg: def fullname(self):

return / print : If print is used inside class
↓
return
doesn't
use brackets
then we can directly call in
instance.

eg: class empl:

def __init__():

def fullname(self):

print()

emp1 = employee (Severthy, Sneha, 23)

emp1.fullname()

→ We use (f "name : {self.first} {self.last},")
f "age : {self.age}"

So f " used when different data grouped
to print.

Class variable : inside class
below class +
above def __init__ () :

eg : class emp :
 school = "HYBE ACADEMY"
 def __init__ (self,) :

Class → blue print

Instance → outside class
 of class

method / → inside class
 attribute

practice :

count → define count as → count = 0
 class variable

4 in init → instance.count += 1

avg → sum () / len ()